

Agent编码规范 约束体系 使用指南

适配所有主流Agent智能体工具的编码约束文件包

本文档是 Agent 编码规范约束文件包的使用指南，详细说明如何将编码规范从"人读文档"转化为"Agent 可执行规则"，覆盖 8 种主流 Agent 工具的入口文件适配、27 个规则文件的部署方法、三层约束执行机制（指令文件 → 分语言规则 → 自动化工具链），以及四阶段落地实施路线。

DOCUMENT INFO

文档版本 V1.0

发布日期 2026年08月

文档类型 使用指南 / 实施方案

目录

Agent编码规范约束体系概述	3
1.1 设计目标	3
1.2 约束执行机制	3
1.3 规则优先级	3
1.4 文件包结构	5
Agent 工具适配方案	6
2.1 支持的工具列表	6
2.2 入口文件设计原理	6
2.3 部署流程	6
2.4 多工具共存场景	8
2.5 规则更新与维护	8
核心规则详解	8
3.1 安全红线	8
3.2 命名规范	9
3.3 函数设计约束	9
3.4 错误处理约束	10
3.5 数据库操作约束	10
3.6 完成检查清单	10
自动化工具链配置	11
4.1 配置文件清单	11
4.2 工具集成优先级	11
4.3 pre-commit 集成	11
4.4 CI/CD 集成建议	13
4.5 各语言 lint 规则集	13
规范落地实施路线	14
5.1 四阶段实施路线	14
5.2 第一阶段：工具先行（1-2 周）	14

5.3 第二阶段：流程保障（2-4 周） 14

5.4 第三阶段：规范覆盖（1-2 月） 15

5.5 第四阶段：持续运营（长期） 15

5.6 关键原则 15

本文件包是一套可被所有主流 AI Agent 编码工具（Claude Code、ZCode、OpenCode、Cursor、Cline、Windsurf、GitHub Copilot 等）直接导入使用的编码开发规范约束体系。它将传统的人读编码规范文档转化为机器可读、Agent 可执行的约束规则文件，使 AI Agent 在生成、修改、审查代码时能够自动遵守预设的编码标准。

1.1 设计目标

传统编码规范文档存在三个核心问题：一是人读文档无法直接约束 Agent 行为；二是各 Agent 工具配置格式不统一，需要重复编写规则；三是规范停留在文档层面，缺乏自动化工具链保障。本约束体系针对这三个问题设计：

- 统一入口**：一套规则，适配所有主流 Agent 工具的入口文件格式
- 机器可读**：规则以 DO/DON'T 格式编写，Agent 可直接理解和执行
- 代码示例**：每条规则附 DO/DON'T 代码示例，Agent 可模式匹配
- 自动化保障**：配套 lint / format / pre-commit 工具配置，工具链兜底
- 分语言粒度**：通用规则 + 分语言规则双层结构，覆盖六大语言

1.2 约束执行机制

本体系通过三层机制约束 Agent 行为：

第一层：Agent 指令文件

各工具的入口文件（CLAUDE.md / .cursorrules 等）在 Agent 启动时自动读取，包含安全红线、命名规范、函数设计、错误处理等核心规则。Agent 在生成代码时将这些规则作为系统提示的一部分。

第二层：分语言规则文件

rules/ 目录下按语言拆分的规则文件提供更详细的约束。Agent 在编写特定语言代码时按需加载对应文件，获取该语言的命名、格式、类型、错误处理、测试等完整规范，含 DO/DON'T 代码示例。

第三层：自动化工具配置

configs/ 目录下的 .editorconfig、ruff.toml、eslint.config.js、golangci.yml 等配置文件，通过 lint 工具和 pre-commit hooks 在代码提交前自动检查和修复格式问题。这是最后一道防线，即使 Agent 生成的代码不完全符合规范，工具链也能兜底。

1.3 规则优先级

Agent 在处理规则冲突时按以下优先级执行：

- **1. 安全红线**（AGENTS.md §1）— 不可违反，违反即终止
- **2. 项目级配置**（pyproject.toml / eslint.config.js 等）— 优先于本文件
- **3. AGENTS.md 主规则** — 通用约束
- **4. rules/ 分语言规则** — 语言级约束
- **5. configs/ 工具配置** — 自动化工具链配置

1.4 文件包结构

完整文件包包含 27 个文件，约 4000 行规则，结构如下：

文件/目录	类型	说明
AGENTS.md	主规则	通用约束中枢，所有工具读取
CLAUDE.md	入口	Claude Code 自动读取
.cursorrules	入口	Cursor IDE 自动读取
.clinerules	入口	Cline 插件自动读取
.windsurfrules	入口	Windsurf IDE 自动读取
copilot-instructions.md	入口	GitHub Copilot 读取
zcode-rules.md	入口	ZCode Agent 读取
opencode-rules.md	入口	OpenCode Agent 读取
rules/00-general.md	规则	通用编码原则（命名/格式/注释/版本控制）
rules/01-python.md	规则	Python 规范（PEP 8 + Google Style）
rules/02-java.md	规则	Java 规范（Google Java Style）
rules/03-javascript.md	规则	JS/TS 规范（Airbnb + Google TS）
rules/04-go.md	规则	Go 规范（Effective Go）
rules/05-rust.md	规则	Rust 规范（API Guidelines）
rules/06-cpp.md	规则	C/C++ 规范（Google C++ Style）
rules/07-sql.md	规则	SQL 编写规范
rules/08-database.md	规则	数据库设计规范
rules/09-nosql.md	规则	NoSQL 规范（MongoDB/Redis/ES）
rules/10-code-review.md	规则	Code Review 与质量保障
configs/.editorconfig	配置	编辑器统一格式
configs/ruff.toml	配置	Python lint + format
configs/eslint.config.js	配置	JS/TS lint
configs/.prettierrc	配置	JS/TS 格式化
configs/golangci.yml	配置	Go lint
configs/clippy.toml	配置	Rust lint
configs/.pre-commit-config.yaml	配置	Git pre-commit hooks

表 1-1 Agent 编码规范约束文件包完整结构

Chapter 2 / 工具适配

Agent 工具适配方案

02

本章详述本规则包如何适配各主流编码工具，包括入口文件格式、规则加载机制和部署方法。

Agent

2.1 支持的工具列表

工具	入口文件	加载机制	部署方式
Claude Code	CLAUDE.md	自动读取项目根目录	复制到项目根
ZCode (Z.ai)	zcode-rules.md	Agent 配置读取	复制到项目根
OpenCode	opencode-rules.md	Agent 配置读取	复制到项目根
Cursor	.cursorrules	IDE 自动读取	复制到项目根
Cline	.clinerules	插件自动读取	复制到项目根
Windsurf	.windsurfrules	IDE 自动读取	复制到项目根
GitHub Copilot	copilot-instructions.md	读取 .github/ 目录	复制到 .github/
Aider / 通用	AGENTS.md	Convention file	复制到项目根

表 2-1 支持的 Agent 编码工具及适配方案

2.2 入口文件设计原理

各工具的入口文件采用"核心规则内联 + 详细规则引用"的双层设计：

核心规则内联

每个入口文件内联了安全红线、命名规范、函数设计、错误处理、数据库操作等最核心的规则。这样即使 Agent 不读取 rules/ 目录，也能遵守基本约束。入口文件控制在 80-120 行以内，避免 token 消耗过大。

详细规则引用

入口文件通过"语言规则索引"表格指向 rules/ 目录下的分语言规则文件。Agent 在编写特定语言代码时按需加载对应文件，获取更详细的规范和代码示例。这种设计平衡了"始终加载核心规则"与"按需加载详细规则"的需求。

2.3 部署流程

以 Claude Code 为例的部署流程：

```
# 1. 复制入口文件和主规则到项目根目录
cp AGENTS.md /your-project/
cp CLAUDE.md /your-project/

# 2. 复制规则目录
cp -r rules/ /your-project/

# 3. 按需复制工具配置
cp configs/.editorconfig /your-project/
cp configs/.pre-commit-config.yaml /your-project/

# Python 项目
cp configs/ruff.toml /your-project/

# JS/TS 项目
cp configs/eslint.config.js /your-project/
cp configs/.prettierrc /your-project/

# Go 项目
cp configs/golangci.yml /your-project/

# Rust 项目
cp configs/clippy.toml /your-project/

# 4. 启用 pre-commit
pip install pre-commit
pre-commit install

# 5. 验证 Agent 已读取规则
# 在 Claude Code 中输入: "请列出你读取的编码规范文件"
# Agent 应回复: AGENTS.md, CLAUDE.md, rules/01-python.md 等
```


2.4 多工具共存场景

团队中可能同时使用多种 Agent 工具（如开发者 A 用 Claude Code，开发者 B 用 Cursor）。本体系的设计天然支持多工具共存：

- 所有入口文件可同时存在于项目根目录，互不冲突
- 所有入口文件共享同一套 rules/ 目录和 configs/ 目录
- 核心规则在各入口文件中保持一致，只是格式适配不同工具
- 新增工具只需添加对应的入口文件，不影响现有工具

2.5 规则更新与维护

当编码规范更新时，只需修改 rules/ 目录下对应的规则文件，所有入口文件自动生效（因为它们引用的是同一份规则文件）。如需更新核心规则（如新增安全红线），需同步修改 AGENTS.md 和各入口文件。

建议的更新流程：

- 1. 修改 rules/ 下的分语言规则文件
- 2. 如涉及核心规则，同步更新 AGENTS.md
- 3. 如涉及入口文件内联规则，同步更新各入口文件
- 4. 提交 PR，Code Review 确认规则变更
- 5. 合并后所有 Agent 工具自动生效

核心规则详解

本章详解约束体系中的核心规则设计，包括安全红线、命名规范、函数设计、错误处理和数据库操作约束。

3.1 安全红线

安全红线是 Agent 不可违反的禁止事项，在任何情况下都不允许突破。这些规则基于 OWASP Top 10 和各语言安全最佳实践：

编号	禁止事项	风险	替代方案
S-01	硬编码密钥/密码/Token	密钥泄露	环境变量/密钥管理服务
S-02	eval/exec/os.system 处理用户输入	代码注入	受限解释器/安全API
S-03	SQL字符串拼接	SQL注入	参数化查询/ORM
S-04	空except/catch块	错误吞没	显式处理+日志记录

S-05	生产代码print/console.log	信息泄露	结构化logger
S-06	注释掉的代码残留	可维护性	直接删除
S-07	git push --force未确认	代码丢失	先确认再执行
S-08	修改main/master未确认	生产风险	通过PR合入
S-09	rm -rf未确认	数据丢失	先确认路径再执行
S-10	暴露调试信息	信息泄露	生产关闭debug模式

表 3-1 安全红线清单

3.2 命名规范

命名规范是代码可读性的基础。本体系为六大语言定义了统一的命名规则：

语言	变量/函数	类/类型	常量	文件名
Python	snake_case	PascalCase	UPPER_SNAKE	snake_case.py
Java	camelCase	PascalCase	UPPER_SNAKE	PascalCase.java
JS/TS	camelCase	PascalCase	UPPER_SNAKE	kebab-case.ts
Go	camelCase/Pascal	PascalCase	PascalCase	snake_case.go
Rust	snake_case	PascalCase	UPPER_SNAKE	snake_case.rs
C/C++	snake_case	PascalCase	UPPER_SNAKE	snake_case.cpp

表 3-2 六大语言命名规范总表

3.3 函数设计约束

Agent 生成的函数必须满足以下量化约束：

约束项	限制值	说明
最大行数	50行	超过需拆分
最大参数数	4个	超过用对象封装
最大嵌套层数	3层	超过用提前返回
最大圈复杂度	15	超过需重构
命名风格	动宾结构	getUserById / calculateTotal

表 3-3 函数设计量化约束

3.4 错误处理约束

Agent 生成的错误处理代码必须遵守以下规则：

规则	说明	示例
显式处理	不吞异常，所有错误必须处理	try/except + logger
保留上下文	传播错误时保留原始信息	raise from err / %w
区分类型	业务错误vs系统错误	BusinessException / SystemException
快速失败	输入校验在入口完成	validate() → fail fast
用户友好	面向用户消息不含技术细节	"操作失败" 非 "NullPointerException"

表 3-4 错误处理约束规则

3.5 数据库操作约束

Agent 生成的数据库操作代码必须遵守以下安全规则：

规则	说明	违规示例	正确示例
参数化查询	禁止SQL拼接	"WHERE id=" + id	"WHERE id=?"
显式列名	禁止SELECT *	SELECT *	SELECT id, name
必须LIMIT	防止全表扫描	SELECT ...	SELECT ... LIMIT 100
事务保护	多行写入用事务	多条独立INSERT	BEGIN/COMMIT
连接释放	用后必须关闭	忘记close()	try-finally / with
索引覆盖	WHERE/JOIN走索引	无索引列查询	EXPLAIN验证

表 3-5 数据库操作安全约束

3.6 完成检查清单

Agent 在报告任务完成前必须逐项确认以下检查清单。这是约束体系的最后一道防线，确保 Agent 交付的代码符合规范：

- [01] 代码符合命名/格式/结构要求
- [02] 已运行lint且无error级别问题
- [03] 已运行类型检查且通过（如适用）
- [04] 已运行相关测试且全部通过
- [05] 无硬编码密钥/密码/路径
- [06] 无调试输出语句残留（print/console.log）
- [07] 新增依赖已加入依赖清单文件
- [08] 公共API有文档注释
- [09] 变更说明已写入commit message

Chapter 4 / 工具配置

自动化工具链配置

04

本章详述 `configs/` 目录下的自动化工具配置文件，这些配置是规则体系的第三层保障——通过 lint、format 和 pre-commit hooks在代码提交前自动检查和修复问题。

4.1 配置文件清单

配置文件	功能	适用语言	工具
<code>.editorconfig</code>	编辑器基础格式统一	全部	IDE原生
<code>ruff.toml</code>	lint + format	Python	Ruff
<code>eslint.config.js</code>	lint	JS/TS	ESLint
<code>.prettierrc</code>	格式化	JS/TS	Prettier
<code>golangci.yml</code>	lint	Go	golangci-lint
<code>clippy.toml</code>	lint	Rust	Clippy
<code>.pre-commit-config.yaml</code>	Git提交前检查	全部	pre-commit

表 4-1 自动化工具配置文件清单

4.2 工具集成优先级

Agent 在处理代码格式时应按以下优先级使用工具：

- 1. 优先使用项目已有的工具配置
- 2. 其次使用 `configs/` 目录下的配置模板
- 3. 最后回退到 AGENTS.md 中的文字规则

4.3 pre-commit 集成

`.pre-commit-config.yaml` 是整个自动化工具链的中枢。它在 `Git` `commit` 执行前自动运行所有语言的 `lint` 和 `format` 工具，任何检查失败都会阻止提交。配置文件集成了以下检查：

检查类型	工具	覆盖范围
基础检查	pre-commit-hooks	空白/换行/大文件/私钥/分支保护
Python	ruff + mypy	lint + format + 类型检查
JS/TS	eslint + prettier	lint + format
Go	gofmt + golangci-lint	格式 + lint

Rust	fmt + clippy	格式 + lint
C/C++	clang-format	格式
安全扫描	bandit + detect-secrets	Python安全 + 密钥检测

表 4-2 pre-commit 集成的检查项

4.4 CI/CD 集成建议

除了 pre-commit 在本地拦截问题外，CI/CD 流水线应在 PR 提交时运行完整的检查链。以下是推荐的 CI 流水线配置：

门禁阈值

检查项	阈值	失败动作
Lint errors	0	阻止合并
Type errors	0	阻止合并
Test failures	0	阻止合并
Coverage	>= 70%	阻止合并
Security vulnerabilities	0 Critical	阻止合并
Code duplication	<= 5%	警告
Cyclomatic complexity	<= 15	警告
Function length	<= 50 lines	警告

表 4-3 CI/CD 门禁阈值配置

4.5 各语言 lint 规则集

Python (Ruff)

启用 E/W/F/I/B/C4/UP/N/SIM/TCH/RUF/PL/S/A/C90 规则集，覆盖 pycodestyle、pyflakes、i sort、bugbear、comprehensions、pyupgrade、pep8-naming、simplify、type-checking、security 等。圈复杂度上限 15，测试文件放宽 assert 和魔法数字限制。

JavaScript/TypeScript (ESLint)

启用 js:recommended + tseslint:strict，禁止 any、强制返回类型、禁止未使用变量、强制类型导入、禁止非空断言。禁止 console（允许 warn/error）。React 项目启用 react 和 react-hooks 插件。

Go (golangci-lint)

启用 errcheck/govet/staticcheck/gocritic/revive/gofmt/goimports/misspell/gocyclo/dupl/goconst/unconvert/prealloc/bodyclose/rowserrcheck/sqlclosecheck/gosec。圈复杂度上限 15。

Rust (Clippy)

启用 all/pedantic/nursery/cargo 规则集，圈复杂度上限 25，函数参数上限 6。通过 -D warnings 将所有 warning 提升为 error。

Chapter 5 / 落地实施

规范落地实施路线

05

本章提供从零到完整的规范落地实施路线，分为四个阶段，总计 3-4 个月完成全面落地。核心原则：工具先行，流程保障，文化扎根。

5.1 四阶段实施路线

阶段	时间	目标	关键动作
一：工具先行	1-2周	自动化工具就位	.editorconfig/lint/pre-commit/CI
二：流程保障	2-4周	开发流程嵌入	PR模板/Review清单/覆盖率门禁
三：规范覆盖	1-2月	全团队覆盖	入口文件部署/培训/考核
四：持续运营	长期	文化扎根	Review文化/定期更新/质量报告

表 5-1 四阶段实施路线

5.2 第一阶段：工具先行（1-2 周）

目标：让自动化工具链就位，代码格式和 lint 检查自动化。

- [1] 配置 .editorconfig 统一编辑器格式
- [2] 按语言集成 lint 工具（ruff/eslint/golangci-lint/clippy）
- [3] 配置 .pre-commit-config.yaml 启用 pre-commit hooks
- [4] CI 流水线加入 lint + typecheck + test 检查
- [5] 安装 detect-secrets 防止密钥泄露

5.3 第二阶段：流程保障（2-4 周）

目标：将规范嵌入开发流程，PR 和 Code Review 流程规范化。

- [1] PR 模板规范化（变更原因/内容/测试方法）
- [2] Code Review Checklist 上线
- [3] 覆盖率门禁启用（>= 70%）
- [4] 安全扫描启用（bandit/gosec/npmaudit）
- [5] Conventional Commits 格式校验

5.4 第三阶段：规范覆盖（1-2 月）

目标：Agent 编码规范全面部署，团队成员培训和考核。

- [1] 复制 AGENTS.md 和各工具入口文件到项目根目录
- [2] 复制 rules/ 目录到项目根目录
- [3] 复制 configs/ 中对应语言的配置文件
- [4] 验证各 Agent 工具能正确读取规则
- [5] 团队培训：规范内容 + 工具使用 + Agent 协作
- [6] 考核：每位开发者通过规范知识测试

5.5 第四阶段：持续运营（长期）

目标：Code Review 文化建设，规范持续更新和优化。

- [1] Code Review 文化建设（Review 质量而非数量）
- [2] 定期规范评审与更新（季度）
- [3] 自动化指标大盘（lint 趋势/覆盖率/缺陷率）
- [4] 季度质量报告（Top 问题/改进进展/下季计划）

5.6 关键原则

整个实施过程遵循三个关键原则：

- **工具先行**：没有自动化工具的规范是纸上谈兵。先让工具能自动检查和拦截，再谈规范遵守。
- **流程保障**：没有 Code Review 的规范是形同虚设。通过 PR 流程和门禁阈值确保规范被执行。
- **文化扎根**：没有团队共识的规范难以持续。通过培训、考核和持续运营让规范成为习惯。

本约束体系将传统编码规范从"人读文档"升级为"Agent 可执行规则 + 自动化工具保障"的双层体系。通过在各主流 Agent 工具中部署本规则包，可以确保 AI Agent 生成的代码在命名、格式、结构、安全、测试等维度持续符合团队标准，让 AI 编码助手真正成为可靠的工程协作伙伴。